PROMO FINDER AI

Solution Documentation

Overview

Promo Finder is a Python CLI application that analyses TV microdrama screenplay transcripts and recommends 20–60 second promotional clips. Each recommendation is designed to open with a hook, end on an unresolved cliffhanger, and remain understandable to viewers unfamiliar with the show. It identifies promo moments and cut points only; it does not edit video or add music/overlays.

Pipeline Architecture

Stage	What it does
Parse	Extract timestamped dialogue, camera, action and sound events, including speakers and text.
Timeline	Build shots, beats and dialogue intervals; identify legal cut points.
Generate	Enumerate valid 20–60 second windows using legal boundaries.
Prefilter	Apply deterministic hook, emotion, coherence and duration rules; retain ≤18 candidates per episode.
Evaluate (optional)	Use an LLM to rate hook, cliffhanger, emotion, coherence and standalone clarity, grounded in source quotes.
Score	Combine LLM and deterministic scores: score = α·llm + (1−α)·det; penalise weak hook, cliffhanger or standalone scores.
Dedupe & rank	Remove overlapping candidates, assign A/B/C tiers and export ranked results.

Core Invariants

Rule	Enforcement
Timestamp authority	The LLM never selects timestamps; all times come from parsed source events.
Event grounding	The LLM references event IDs only; unknown IDs are discarded.
Quote fidelity	Hook and cliffhanger quotes must be normalised substrings of source events.
Cut safety	Promos are 20–60 seconds; cuts never fall strictly inside dialogue; segments are chronological and non-overlapping.

Running the Application

Heuristic-only mode (for workflow testing)

Runs offline; approximately 10 seconds for 50 episodes.

```
cd /home/ishant615/Documents/charu/promo_finder_AI
PYTHONPATH=src python3 -m promo_finder run --input data/input --no-llm
```

LLM-assisted mode

Adds story-quality ratings for preselected candidates. Timestamp and cut-point selection remain deterministic.

Process a subset of episodes

```
PYTHONPATH=src python3 -m promo_finder run --input data/input --no-llm --max-episodes 5
```

Outputs

Each run creates a timestamped directory under outputs/:

File	Contents
promos.json	Ranked promos, scores, breakdowns and hook/cliffhanger quotes.

report.md	Human-readable ranked table and promo cards.
run_manifest.json	Provider, model, timing and configuration metadata.
candidates_debug.json	Scored candidates, including rejected candidates.
episodes.json	Parsed episode data.
promos_incremental.json	Promos written after each episode during long runs.
progress.json	Episode completion progress.

During execution, the CLI reports the current episode and remaining count. progress.json is updated after each episode for live progress monitoring.

Promo Result Contents

- Rank, A/B/C tier and score out of 100.
- Segment start/end times in MM:SS.ss and floating-point seconds.
- Verbatim hook and cliffhanger quotes from the transcript.
- Per-dimension scores: hook, cliffhanger, emotion, coherence, standalone clarity and duration.
- Flags, weaknesses and confidence level.

Testing and Quality

- 102 unit tests pass (pytest -q), covering parsing, timestamps, cut points, features, scoring, deduplication, ranking, grounding and output writing.
- The invariant checker reports zero errors: all promos meet duration bounds, dialogue-safe cut rules and verbatim-quote requirements.

Known Limitations

- Scoring weights and thresholds are initial values; no labelled dataset is available for tuning.
- LLM mode takes 60+ minutes for 50 episodes because of provider rate limits and retries.
- Free-tier providers (Groq, OpenRouter) impose strict limits. The evaluator retries up to 50 times with exponential backoff; LLM integration was verified on only one episode, so a full 50-episode LLM run has not been validated.
- Only contiguous, single-segment promos are supported; stitching across scenes or episodes is deferred.
- The application currently provides a CLI; a web UI is deferred.

Configuration

Primary settings are in config/default.yaml and can be overridden through environment variables or a .env file. Provider SDKs are lazy-imported, so heuristic-only execution does not require them to be installed.